Vanishing Point

Project Overview: Phase 1 (Simulator & Seeker) and Phase 2 (PPO Evader)

1. Project in one paragraph

Vanishing Point trains an Evader drone to stay out of a Seeker drone's camera view in a bounded 3D arena (20 × 20 × 10 m, 0.1 s timestep, 600-step episodes). Phase 1 built the simulator, a scripted Seeker, and a random-Evader baseline. Phase 2 trained the Evader with PPO, first plainly and then with a difficulty curriculum, and validated the results across multiple seeds. Stage 2 of the plan (a learning Seeker via MADDPG or self-play) has not started yet.

2. Phase 1: building the simulator and the scripted Seeker

2.1 First code version

The first simulator version was a clean single file with correct FOV geometry and simple velocity-controlled drones, but it had several problems that made its results unreliable: a wall penalty of -10 per step that dwarfed the ±1 visibility reward, a fixed random seed (7) that made every run identical, and a Seeker that simply gave up and span in place after losing the Evader, with a scan pattern that left a blind spot above and below.

Metric	Value (seed 3167782036)
Total reward	426.0
Undetected time	86.2%
Time to first detection	0.0 s (uninformative metric)
Wall touches	4

With this version, the Seeker locked onto the Evader early (by step 80), lost it around step 100 when the Evader drifted 64° off-angle, and then never found it again for the rest of the episode, about 517 of 600 steps hidden. That is a misleading baseline: the high undetected percentage reflects a Seeker that gave up, not an Evader that is hard to catch.

2.2 Fixing the Seeker

The Evader was left random and the Seeker was tuned into a fair opponent before any PPO training began:

Change	Before	After	Why
Wall penalty	-10 every step against a wall	-2 once on first contact	Keeps hiding/being-seen as the main signal
Random seed	Fixed at 7	New each run, logged	Shows real variation, allows replay
Seeker modes	Follow, then spin forever	TRACK, SCAN (4s timeout), SEARCH	Recovers after losing the Evader
Scan pattern	Left/right only	Spin + ±45° tilt	Removes blind spot above/below
Start behaviour	Slow forward patrol	Scan first, then search	Avoids getting stuck at a wall
Metrics	Time to first detection	Time until first loss, longest hidden stretch, wall touches, search restarts	Describes how an episode unfolds

2.3 Results after tuning

Metric	First version	Tuned version (seed 3171674840)
Total reward	426.0	-256.0
Undetected time	86.2%	30.3%
Time until first loss of sight	~8s (inferred)	8.6 s
Longest undetected stretch	~50s (inferred)	8.3 s
Wall touches	4	10
Seeker search restarts	n/a	1

The reward sign flipped because the fixed Seeker now sees the random Evader about 70% of the time, giving PPO a real benchmark to beat instead of a Seeker that could be escaped by accident. A 20-seed headless test confirmed the improvement generalises rather than being a one-off:

Metric (20-seed average)	Old Seeker	New Seeker
Undetected time	56.0%	37.7%
Longest hidden stretch	22.2 s	13.4 s
Worst single stretch	50.0 s	48.9 s

The worst single hidden stretch barely moved (50.0 s to 48.9 s), showing the Seeker's random search waypoints still leave it blind on unlucky seeds — a known weakness carried into Phase 2.

2.4 Phase 1 pros and cons

- Pros: recovers after losing the Evader via SCAN/SEARCH, scans the full 3D space, reproducible seeds, a meaningful opponent and baseline for PPO.
- Cons: random search waypoints still leave long worst-case hidden stretches; the Seeker (1.8 m/s) is slower than the Evader (2.2 m/s) and does not predict movement; drones can overlap with no penalty.

3. Phase 2: training the Evader with PPO

Three controllers were compared against the same tuned scripted Seeker: acting randomly, PPO trained the plain way for 500,000 steps, and PPO trained the same length with a curriculum that starts against a weak Seeker and strengthens it across four stages. Each setup was run across 3 training seeds (1 original + 2 more added later) and evaluated on the same 20 episodes at full difficulty.

3.1 Results per seed

Setup	Seed	Mean return	Undetected	Wall touches
Random baseline	12345	173.3	66.0%	9.25
Plain PPO	577448602 (original)	466.5	89.6%	4.20
Plain PPO	2001	470.5	90.0%	4.55
Plain PPO	2002	472.7	91.0%	9.50
Curriculum PPO	577448602 (original)	458.8	88.7%	2.60
Curriculum PPO	2001	452.6	88.2%	3.00
Curriculum PPO	2002	447.0	87.9%	3.80

3.2 Averaged comparison

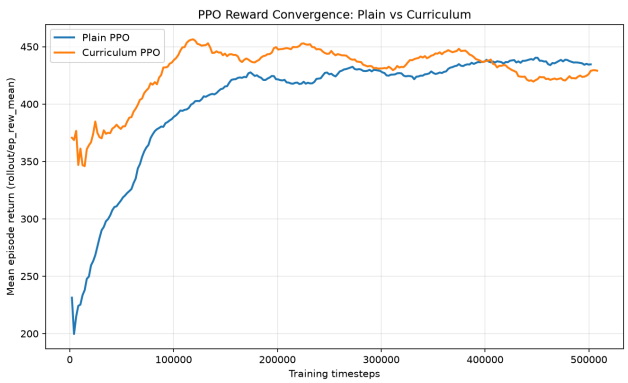
Metric	Random baseline	Plain PPO (avg of 3 seeds)	Curriculum PPO (avg of 3 seeds)
Mean return	173.3	469.9	452.8
Undetected time	66.0%	90.2%	88.3%
Time until first loss	3.5 s	1.7 s	1.8 s
Longest hidden stretch	10.7 s	33.3 s	27.4 s
Wall touches (mean)	9.25	6.08	3.13
Seeker search restarts	1.30	5.55	4.78

PPO is a large jump over random behaviour on every metric: undetected time rises from 66.0% to about 90%, sight is lost almost immediately (1.7-1.8 s versus 3.5 s), and wall touches fall. Between the two PPO variants, the pattern held across all three seeds without exception: plain PPO scored higher every time, and curriculum PPO touched walls less often every time. Averaged out, plain PPO wins on return (469.9 vs 452.8) and undetected time (90.2% vs 88.3%), while curriculum PPO roughly halves wall touches (6.08 vs 3.13). This is a genuine, repeatable trade-off rather than single-seed noise.

Why the trade-off happens: reward only measures undetected time, and the wall penalty fires once per contact rather than scaling with how close or how often the drone grazes a wall, so a policy that clips corners to break line of sight a step or two earlier is not punished much for the risk it takes. Training the whole run at full seeker speed and field of view lets plain PPO find and keep that riskier shortcut, since it never has to unlearn it once discovered. Curriculum training instead spends its early stages against a slow, narrow-FOV seeker, where survival depends on clean navigation rather than desperate manoeuvres near walls; that collision-safe habit is already set by the time stage four brings the seeker up to full difficulty, so the agent keeps navigating conservatively even though it could now afford to cut corners. The training logs support this: under curriculum, ep_rew_mean recovers within a few thousand timesteps after each difficulty increase instead of collapsing, and the action-distribution std grows more smoothly, whereas the plain runs show noisier, less monotonic reward climbs through roughly the first 100,000-150,000 timesteps before settling. Wall-touch variance across seeds tells the same story: curriculum stays in a tight band (2.60-3.80), while plain PPO ranges far wider (4.20-9.50), including one seed (2002) that settled on a distinctly wall-heavy strategy. In short, plain PPO chases the highest score the reward function allows; curriculum PPO trades a few points of that score for a policy that behaves consistently regardless of seed.

3.3 Reward convergence

The plot below shows the training-time reward curve for one seed of each setup. Curriculum training starts far higher since its early stages face an easy, slow, narrow-sighted Seeker — an early lead there is expected, not meaningful. Both curves reach a similar plateau by the end, and curriculum PPO dips slightly below plain PPO in the final stretch, which lines up with curriculum stage 4 (full difficulty) arriving late and the model having little time to adapt to the harder Seeker.



3.4 Independent tests

Automated tests were added to check behaviour that training runs alone could not verify: the FOV boundary, Seeker mode transitions, the one-time wall penalty, valid resets, correct episode length, and the absence of NaN/infinite values. All eleven test cases pass.

Test area	Result
FOV boundary (visible/not visible at edges and range)	Pass
Seeker mode transitions (visible to TRACK, timeout to SEARCH)	Pass
Wall penalty fires once per contact, not once per step in contact	Pass
Reset returns a valid, bounded observation	Pass
Episode truncates at exactly 600 steps	Pass
No NaN or infinite values across random episodes	Pass

3.5 Phase 2 pros and cons

- Pros: PPO massively outperforms random behaviour; the Evader loses the Seeker almost immediately and touches walls far less; curriculum training is confirmed (3 seeds) to trade a small amount of score for materially fewer wall collisions.
- Cons: the Seeker is still slower than the Evader, so part of the score may reflect out-running rather than pure evasive skill; the Evader's observation excludes the Seeker's current mode; only 3 seeds have been tested, and only at full difficulty for the final comparison.

4. Which model to use, and current status

Plain PPO (trained on seed 577448602, saved as models/ppo_500k) remains the headline result and the strongest average performer. The curriculum model is a validated secondary finding, not a replacement: it consistently sacrifices some peak score for steadier, more wall-aware behaviour, which may matter more once Stage 2 introduces a second learning agent. With three seeds, a convergence plot and a passing test suite in place, Phase 2 (formulation, environment, PPO training, curriculum, evaluation, and testing) is complete against the implementation plan. Stage 2, replacing the scripted Seeker with a learning Seeker under MADDPG or self-play, has not been started.

5. Appendix: how to reproduce everything from scratch

All commands assume the 'vp' conda environment and that the terminal is already in the project folder containing vanishing_point_ppo.py, plot_convergence.py and test_vanishing_point.py.

5.1 Environment

```
conda activate vp
PY=$CONDA_PREFIX/bin/python
$PY -m pip install tensorboard matplotlib pytest
```

5.2 Random baseline

```
$PY vanishing_point_ppo.py random --episodes 20
```

5.3 Train the three seeds, plain and curriculum (6 runs)

```
$PY vanishing_point_ppo.py train --timesteps 500000 --seed 577448602 --model models/ppo_500k --tensorboard-log logs/
$PY vanishing_point_ppo.py train --curriculum --timesteps 500000 --seed 577448602 --model models/ppo_curr --tensorboard-log logs/
```

```
$PY vanishing_point_ppo.py train --timesteps 500000 --seed 2001 --model models/ppo_plain_s2001 --tensorboard-log logs/
$PY vanishing_point_ppo.py train --timesteps 500000 --seed 2002 --model models/ppo_plain_s2002 --tensorboard-log logs/
$PY vanishing_point_ppo.py train --curriculum --timesteps 500000 --seed 2001 --model models/ppo_curr_s2001 --tensorboard-log logs/
$PY vanishing_point_ppo.py train --curriculum --timesteps 500000 --seed 2002 --model models/ppo_curr_s2002 --tensorboard-log logs/
```

Run each training command one at a time and let it finish (roughly 4-5 minutes each) before starting the next.

5.4 Evaluate all six models

```
$PY vanishing_point_ppo.py eval --model models/ppo_500k --episodes 20
$PY vanishing_point_ppo.py eval --model models/ppo_curr --episodes 20
$PY vanishing_point_ppo.py eval --model models/ppo_plain_s2001 --episodes 20
$PY vanishing_point_ppo.py eval --model models/ppo_plain_s2002 --episodes 20
$PY vanishing_point_ppo.py eval --model models/ppo_curr_s2001 --episodes 20
$PY vanishing_point_ppo.py eval --model models/ppo_curr_s2002 --episodes 20
```

5.5 Watch the trained Evader (optional)

```
$PY vanishing_point_ppo.py eval --model models/ppo_500k --episodes 3 --render
```

5.6 Reward convergence plot

```
ls logs/
```

Match the plain seed 577448602 and curriculum seed 577448602 runs to their PPO_N folder names (they are the first two trained), then run:

```
$PY plot_convergence.py --plain logs/PPO_1 --curriculum logs/PPO_3 --out convergence.png
```

5.7 Independent test suite

```
$PY -m pytest test_vanishing_point.py -v
```

5.8 Next: Stage 2 planning

Once the above is reproduced and confirmed, the project is ready to begin Stage 2: replacing the scripted Seeker with a learning Seeker under a multi-agent Markov game formulation (MADDPG or self-play), plus the previously noted improvements (minimum drone separation, smarter search waypoints biased toward the last-known position, and giving the Evader the Seeker's current mode as an extra observation).